MODULO 4 · LEZIONE 5

Un tool che esegue codice, sotto approvazione

Per la parte 2 del lab di Modulo 4 · Lezione 6

IL TOOL CHE ESEGUE

Un tool che esegue codice Python

```
@function_tool(needs_approval=True)
def esegui_python(codice: str) -> str:
    ...
    processo = subprocess.run(
        [sys.executable, "-c", codice], cwd=RADICE, env=ambiente,
        stdout=subprocess.PIPE, stderr=subprocess.STDOUT,
        text=True, encoding="utf-8", errors="replace", timeout=TIMEOUT)
    ...
    return f"codice di uscita {processo.returncode}\n{testa_e_coda(processo.stdout)}"
```

Un processo

Parte nella cartella dell'ufficio, con il Python del venv. Il modello riceve il codice di uscita e quello che il programma stampa.

Con dei limiti

Si ferma dopo 30 secondi. Oltre 10.000 caratteri di output, l'inizio e la fine, come mini-swe-agent.

Senza recinto

Può leggere ../.env, scrivere, cancellare: per questo needs_approval=True.



IL TOOL CHE ESEGUE

Chi scrive, chi approva, chi lancia

1 Il modello scrive

Il codice è l'argomento codice di una chiamata a esegui_python: un testo.

2 Voi approvate

Dopo che il modello l'ha scritto, prima che parta. Sì o no, per ogni esecuzione.

3 Il tool lancia

Un processo sulla vostra macchina, con i vostri permessi.

> Lo stesso permesso che vi chiede il coding agent, scritto nel codice del nostro assistente.



IL TOOL CHE ESEGUE

Cosa guardare prima del sì

Domanda	Nel codice
Scrive o cancella file?	open con 'w', 'a' o 'x', csv.writer, write_text, to_csv, os.replace, os.rename, os.remove, unlink, shutil
Esce dalla cartella?	.. nei percorsi, percorsi assoluti
Lancia programmi o va in rete?	subprocess, os.system, urllib, socket
Legge l'ambiente?	os.environ: il tool toglie solo la chiave di OpenAI
Il conto è giusto?	con quale colonna unisce i file, quale filtro mette

> Prima di ogni sì, una riga nei vostri appunti su cosa fa il codice. Se non riuscite a scriverla, è no.



L'APPROVAZIONE

Il ciclo di approvazione nell'SDK

```
result = await Runner.run(agente, domanda)
while result.interruptions:
    stato = result.to_state()
    for interruzione in result.interruptions:
        if la_persona_dice_si(interruzione):
            stato.approve(interruzione)
        else:
            stato.reject(interruzione, rejection_message=motivo)
    result = await Runner.run(agente, stato)
```

Si ferma

Prima del tool, Runner.run finisce: le chiamate da approvare sono in result.interruptions.

Decidete

Sì e no si scrivono nello stato del run, con approve e reject.

Riprende

Con lo stato al posto della domanda: esegue i sì e richiama il modello.



L'APPROVAZIONE

Cosa vedete a schermo

```
Il modello vuole chiamare esegui_python.
  1 | import csv
  2 | p='note_spese.csv'; d='dipendenti.csv'
  3 | with open(d, newline='', encoding='utf-8') as f:
  4 |     padova={r['id'] for r in csv.DictReader(f) if r['sede']=='Padova'}
  5 | with open(p, newline='', encoding='utf-8') as f:
  6 |     reader=csv.DictReader(f); fields=reader.fieldnames; rows=list(reader)
  7 | ids=[]
  8 | for r in rows:
  9 |     if r['dipendente_id'] in padova and r['stato']=='in attesa':
 10 |         r['stato']='approvata'; ids.append(r['id'])
 11 | with open(p, 'w', newline='', encoding='utf-8') as f:
 12 |     w=csv.DictWriter(f, fieldnames=fields); w.writeheader(); w.writerows(rows)
 13 | print('Aggiornate', len(ids), 'righe:', ', '.join(ids))
Eseguo? [s = sì, n = no, e = esci e riprendi dopo] n
```

Qui n: la riga 11 riscrive note_spese.csv (riga 2). Ogni tasto diverso da s, Invio compreso, vale no; dopo il no si chiede il motivo, e Invio vuol dire nessun motivo.



L'APPROVAZIONE

Cosa riceve il modello dopo un no

> L'output della chiamata rifiutata, il testo di default dell'SDK
>
> ```
> Tool execution was not approved.
> ```

> Con un motivo, il modello riceve quel testo al posto del default
>
> ```
> stato.reject(interruzione, rejection_message="No: i file dell'ufficio non si modificano.")
> ```

È tutto quello che il modello sa: non sa chi ha detto no, né perché, se il testo non lo dice.



L'APPROVAZIONE

Lo stesso controllo nel coding agent

.claude/settings.json, nella cartella del progetto

```
{
  "permissions": {
    "ask": ["Bash(python *)"],
    "deny": ["Bash(rm *)"]
  }
}
```

ask chiede ogni volta, come needs_approval=True.

deny non fa partire il comando; per rm prova.txt il modello riceve «Permission to use Bash with command rm prova.txt has been denied.»

Una regola guarda il testo del comando: Bash(rm *) ferma rm -rf build/, non /bin/rm -rf build/.



Calcolare invece di leggere

Prossimo blocco, 40 minuti: esegui_python acceso, un compito sul CSV grande con il codice letto prima di ogni sì, poi un compito da rifiutare.

Per esteso, dopo il lab: la dispensa di Modulo 4 · Lezione 5.